RustCompCert

A Verified and Verifying Compiler for a Sequential Subset of Rust

Jinhua Wu [1] Yuting Wang [1] Liukun Yu [1] Linglong Meng [2]

[1] Shanghai Jiao Tong University, China
[2] University of Minnesota, USA

Rust Verification Workshop
Turin, Italy
April 14, 2026

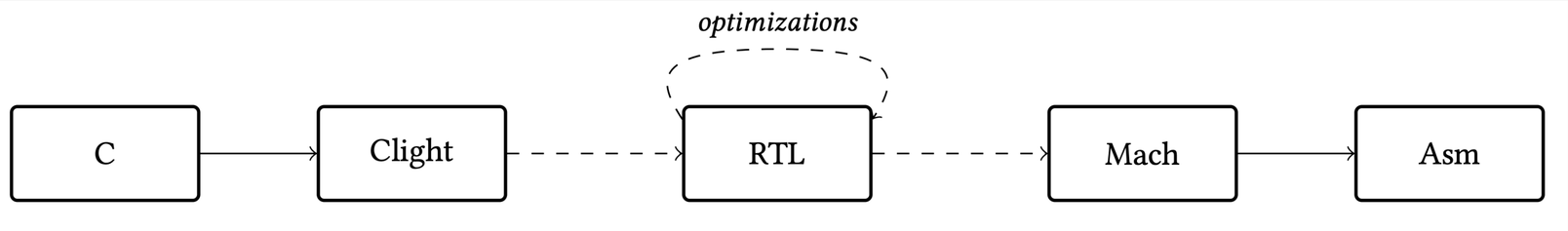
In this talk

- Background, motivation and our contributions
- Demo: traversal of a linked list
- Roadmap of RustCompCert:
  - Introduce our high-level design of RustCompCert
- Future plan

Introduction of CompCert

Introduction of CompCert

- The state-of-the-art formally verified C compiler

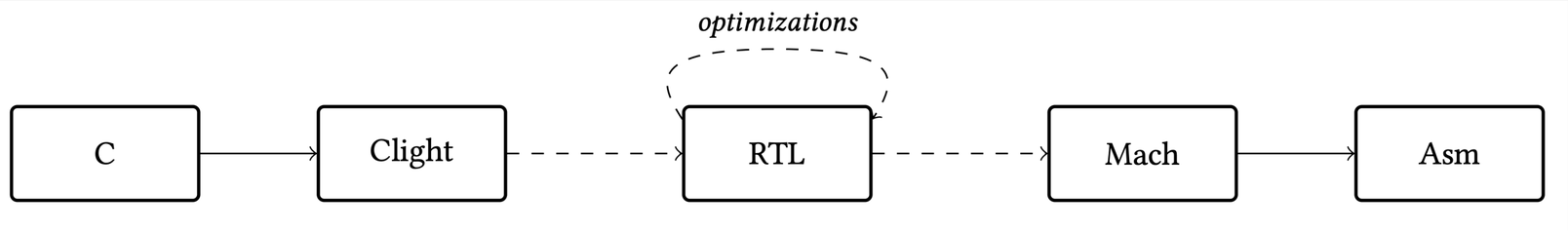


Verified compilation chain of CompCert

Introduction of CompCert

- The state-of-the-art formally verified C compiler
- Compiler correctness—semantics preservation theorem:

$$\forall P_C\,,\, P_{\text{Asm}}\,,\, \text{CompCert}(P_C) = P_{\text{Asm}} \;\Rightarrow\; \text{Beh}(P_C) \supseteq \text{Beh}(P_{\text{Asm}})$$



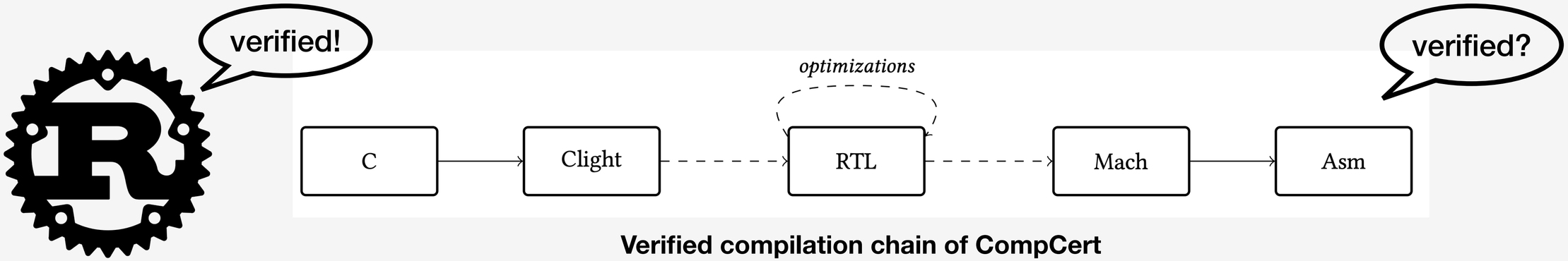
Verified compilation chain of CompCert

Introduction of CompCert

- The state-of-the-art formally verified C compiler
- Compiler correctness—semantics preservation theorem:

$$\forall P_C\,,\, P_{\text{Asm}}\,,\, \text{CompCert}(P_C) = P_{\text{Asm}} \;\Rightarrow\; \text{Beh}(P_C) \supseteq \text{Beh}(P_{\text{Asm}})$$

- Motivation: in Rust, how to ensure properties verified at the source level are still valid at the target level?



Verified compilation chain of CompCert

Existing work

Existing work

- Formalization and verification of Rust borrow checker
  - RustBelt[Jung et al. 2017] and Oxide[Weiss et al. 2019]: (semantic) type systems
  - Aeneas[Ho et al. 2024]: symbolic execution of LLBC
  - a-mir-formality[rust team]: formalize Rust's checking rules

Existing work

- Formalization and verification of Rust borrow checker
  - RustBelt[Jung et al. 2017] and Oxide[Weiss et al. 2019]: (semantic) type systems
  - Aeneas[Ho et al. 2024]: symbolic execution of LLBC
  - a-mir-formality[rust team]: formalize Rust's checking rules
- Formalization and verification of Rust compilation passes
  - Formalization of Rust Drop Elaboration[Fukala et al. 2024]

Existing work

- Formalization and verification of Rust borrow checker
  - RustBelt[Jung et al. 2017] and Oxide[Weiss et al. 2019]: (semantic) type systems
  - Aeneas[Ho et al. 2024]: symbolic execution of LLBC
  - a-mir-formality[rust team]: formalize Rust's checking rules
- Formalization and verification of Rust compilation passes
  - Formalization of Rust Drop Elaboration[Fukala et al. 2024]
- **How to formalize Rust’s compilation passes and borrow checking algorithm in a verified compiler framework?**

This work: RustCompCert

Towards a formally verified Rust compiler

This work: RustCompCert

Towards a formally verified Rust compiler

- Our work is distinguished by three key aspects:

This work: RustCompCert

Towards a formally verified Rust compiler

- Our work is distinguished by three key aspects:
  - A prototype verified compiler targeting a safe subset of Rust

This work: RustCompCert

Towards a formally verified Rust compiler

- Our work is distinguished by three key aspects:
  - A prototype verified compiler targeting a safe subset of Rust
  - A formalized Polonius-based borrow checking by reusing CompCert's dataflow analysis framework

This work: RustCompCert

Towards a formally verified Rust compiler

- Our work is distinguished by three key aspects:
  - A prototype verified compiler targeting a safe subset of Rust
  - A formalized Polonius-based borrow checking by reusing CompCert's dataflow analysis framework
  - A scheme of supporting interoperation between safe and unsafe code, based on techniques of verified compositional compilation

This work: RustCompCert

Towards a formally verified Rust compiler

- Our work is distinguished by three key aspects:
  - A prototype verified compiler targeting a safe subset of Rust
  - A formalized Polonius-based borrow checking by reusing CompCert's dataflow analysis framework
  - A scheme of supporting interoperation between safe and unsafe code, based on techniques of verified compositional compilation

What we do not aim for:

- This work does not aim to verify rustc

Demo: traversal of a linked list

Our roadmap of RustCompCert

Our roadmap of RustCompCert

- Step 1: a **verified** compiler for a safe subset of Rust (✅)

Our roadmap of RustCompCert

- Step 1: a **verified** compiler for a safe subset of Rust (✅)
- Step 2: a **verified** and **verifying** compiler for an ownership subset of Rust (✅)

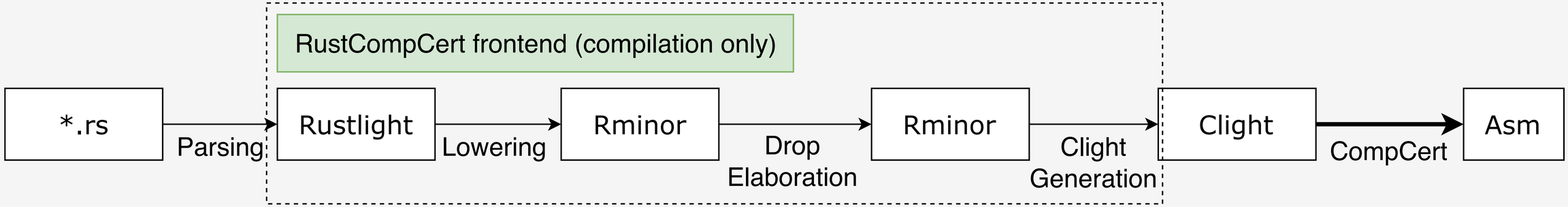
Our roadmap of RustCompCert

- Step 1: a **verified** compiler for a safe subset of Rust (✅)
- Step 2: a **verified** and **verifying** compiler for an ownership subset of Rust (✅)
- Step 3: a **verified** and **verifying** compiler for a safe subset of Rust (🧑‍💻)

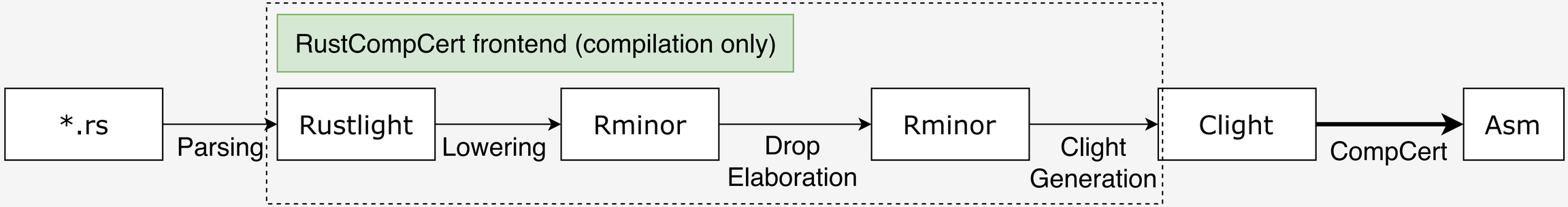
Our roadmap of RustCompCert

- Step 1: a **verified** compiler for a safe subset of Rust (✅)
- Step 2: a **verified** and **verifying** compiler for an ownership subset of Rust (✅)
- Step 3: a **verified** and **verifying** compiler for a safe subset of Rust (👩‍💻)
- Step 4: a **compositional, verified** and **verifying** compiler for a safe subset of Rust (👩‍💻)

A verified compiler for a subset of Rust

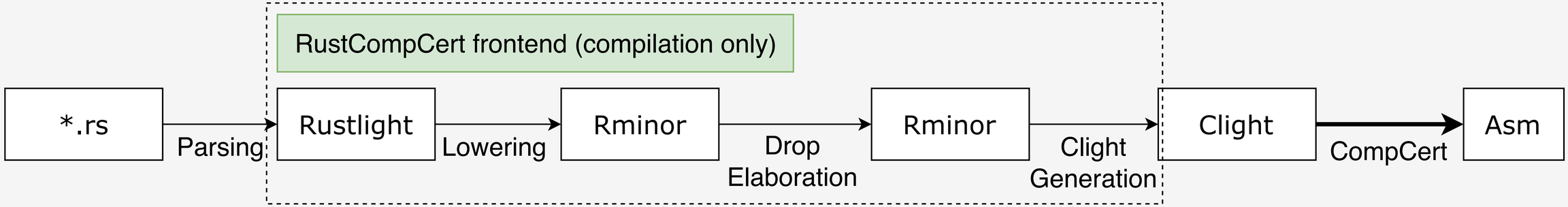


A verified compiler for a subset of Rust




```
// code snippet from iterate_until_consume
match *l {
  List::Nil => {
    return;
  }
  List::Cons(node) => {
    if node.val == x {
      *l = List::Nil;
    } else {
      l = node.next;
    }
  }
}
```

A verified compiler for a subset of Rust




```
// code snippet from iterate_until_consume
match *l {
  List::Nil => {
    return;
  }
  List::Cons(node) => {
    if node.val == x {
      *l = List::Nil;
    } else {
      l = node.next;
    }
  }
}
```

Parsing

```
// Rustlight code, after parsing
if (discriminant_eq(*l, Nil)) {
  ... // omitted
} else {
  let node : Node in {
    node = move (*l as Cons);
    if (node.val == x) {
      let _32 : List in {
        _32 = List::Nil(tt);
        *l = move _32;
      }
    } else {
      l = move node.next;
    }
  }
}
```

A verified compiler for a subset of Rust

A verified compiler for a subset of Rust

Identifiers	$id, n \in \mathbb{N}$
Places	$p \in P ::= id \mid *p \mid p.id \mid p\ as\ id$
Mutability	$\omega ::= \texttt{mut} \mid \texttt{shr}$
Regions	$\rho, r \in R ::= \mathbb{N}$
Base Types	$\tau^B ::= \texttt{unit} \mid \texttt{bool} \mid \texttt{i32} \mid \texttt{f32} \mid \ldots$
Field Lists	$\varphi ::= [(id_1, \tau_1); ...; (id_n, \tau_n)]$
Types	$\tau \in \mathcal{T} ::= \tau^B \mid \texttt{Box}\ \tau \mid \&\rho\ \omega\ \tau$
	$\mid \texttt{struct}\langle\overrightarrow{\rho}\rangle(id, \varphi) \mid \texttt{enum}\langle\overrightarrow{\rho}\rangle(id, \varphi)$
	$\mid \texttt{fn}\ \langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(\tau_1, ..., \tau_n) \to \tau$
Constants	$c ::= () \mid \texttt{true} \mid \texttt{false} \mid n \mid \ldots$

Pure Expressions	$e^P ::= c \mid p \mid \&\rho\ \omega\ p \mid \texttt{uop}\ e \mid e_1\ \texttt{bop}\ e_2$
	$\mid \texttt{discriminant_eq}(p, id)$
Expressions	$e ::= e^P \mid \texttt{move}\ p$
Statements	$s ::= p := e \mid p := id_{\texttt{enum}} :: id_\varphi(e) \mid p := \texttt{Box}(e)$
	$\mid p := id_f(\overrightarrow{e}) \mid \texttt{return}(p)$
	$\mid \texttt{let}\ x : \tau\ \texttt{in}\ s \mid s_1; s_2 \mid \text{if}\ e^P\ \texttt{then}\ s_1\ \texttt{else}\ s_2$
	$\mid \texttt{loop}\ s \mid \texttt{break} \mid \texttt{continue}$
Variable Declarations	$dcl ::= id_1 : \tau_1, ..., id_n : \tau_n$
Internal Functions	$fd ::= \texttt{fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau\{dcl_2; s\}$
External Functions	$fe ::= \texttt{extern}\ \texttt{fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau$
Functions	$f ::= fd \mid fe$
Modules	$Mods ::= \{(id_1, f_1), ..., (id_n, f_n)\}$

Syntax of Rustlight

A verified compiler for a subset of Rust

Identifiers	$id, n \in \mathbb{N}$
Places	$p \in P ::= id \mid *p \mid p.id \mid p\ as\ id$
Mutability	$\omega ::= \texttt{mut} \mid \texttt{shr}$
Regions	$\rho, r \in R ::= \mathbb{N}$
Base Types	$\tau^B ::= \texttt{unit} \mid \texttt{bool} \mid \texttt{i32} \mid \texttt{f32} \mid \ldots$
Field Lists	$\varphi ::= [(id_1, \tau_1); ...; (id_n, \tau_n)]$
Types	$\tau \in \mathcal{T} ::= \tau^B \mid \texttt{Box}\ \tau \mid \&\rho\ \omega\ \tau$
	$\mid \texttt{struct}\langle\overrightarrow{\rho}\rangle(id, \varphi) \mid \texttt{enum}\langle\overrightarrow{\rho}\rangle(id, \varphi)$
	$\mid \texttt{fn}\ \langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(\tau_1, ..., \tau_n) \to \tau$
Constants	$c ::= () \mid \texttt{true} \mid \texttt{false} \mid n \mid \ldots$

Pure Expressions	$e^P ::= c \mid p \mid \&\rho\ \omega\ p \mid \texttt{uop}\ e \mid e_1\ \texttt{bop}\ e_2$
	$\mid \texttt{discriminant_eq}(p, id)$
Expressions	$e ::= e^P \mid \texttt{move}\ p$
Statements	$s ::= p := e \mid p := id_{\texttt{enum}} :: id_\varphi(e) \mid p := \texttt{Box}(e)$
	$\mid p := id_f(\overrightarrow{e}) \mid \texttt{return}(p)$
	$\mid \texttt{let}\ x : \tau\ \texttt{in}\ s \mid s_1; s_2 \mid \text{if}\ e^P\ \texttt{then}\ s_1\ \texttt{else}\ s_2$
	$\mid \texttt{loop}\ s \mid \texttt{break} \mid \texttt{continue}$
Variable Declarations	$dcl ::= id_1 : \tau_1, ..., id_n : \tau_n$
Internal Functions	$fd ::= \texttt{fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau\{dcl_2; s\}$
External Functions	$fe ::= \texttt{extern fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau$
Functions	$f ::= fd \mid fe$
Modules	$Mods ::= \{(id_1, f_1), ..., (id_n, f_n)\}$

Syntax of Rustlight

- Supported: struct and enum, heap allocation (by Box) and reference

A verified compiler for a subset of Rust

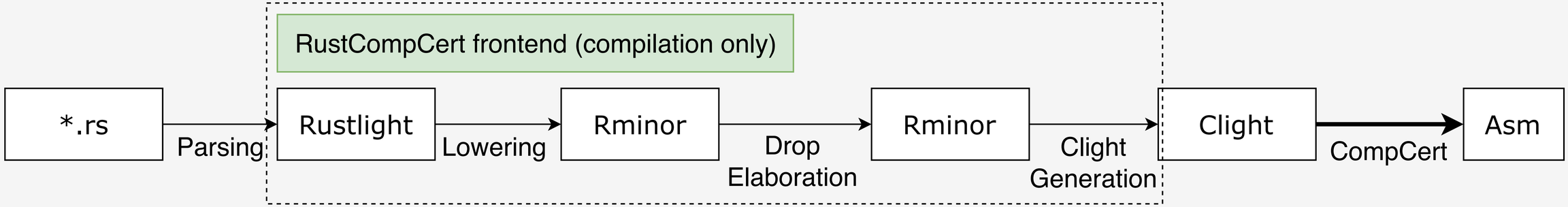
Identifiers	$id, n \in \mathbb{N}$
Places	$p \in P ::= id \mid *p \mid p.id \mid p\ as\ id$
Mutability	$\omega ::= \texttt{mut} \mid \texttt{shr}$
Regions	$\rho, r \in R ::= \mathbb{N}$
Base Types	$\tau^B ::= \texttt{unit} \mid \texttt{bool} \mid \texttt{i32} \mid \texttt{f32} \mid \ldots$
Field Lists	$\varphi ::= [(id_1, \tau_1); ...; (id_n, \tau_n)]$
Types	$\tau \in \mathcal{T} ::= \tau^B \mid \texttt{Box}\ \tau \mid \&\rho\ \omega\ \tau$
	$\mid \texttt{struct}\langle\overrightarrow{\rho}\rangle(id, \varphi) \mid \texttt{enum}\langle\overrightarrow{\rho}\rangle(id, \varphi)$
	$\mid \texttt{fn}\ \langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(\tau_1, ..., \tau_n) \to \tau$
Constants	$c ::= () \mid \texttt{true} \mid \texttt{false} \mid n \mid \ldots$

Pure Expressions	$e^P ::= c \mid p \mid \&\rho\ \omega\ p \mid \texttt{uop}\ e \mid e_1\ \texttt{bop}\ e_2$
	$\mid \texttt{discriminant_eq}(p, id)$
Expressions	$e ::= e^P \mid \texttt{move}\ p$
Statements	$s ::= p := e \mid p := id_{\texttt{enum}} :: id_\varphi(e) \mid p := \texttt{Box}(e)$
	$\mid p := id_f(\overrightarrow{e}) \mid \texttt{return}(p)$
	$\mid \texttt{let}\ x : \tau\ \texttt{in}\ s \mid s_1; s_2 \mid \text{if}\ e^P\ \texttt{then}\ s_1\ \texttt{else}\ s_2$
	$\mid \texttt{loop}\ s \mid \texttt{break} \mid \texttt{continue}$
Variable Declarations	$dcl ::= id_1 : \tau_1, ..., id_n : \tau_n$
Internal Functions	$fd ::= \texttt{fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau\{dcl_2; s\}$
External Functions	$fe ::= \texttt{extern fn}\ id\langle\overrightarrow{\rho}, \overrightarrow{\rho_1 : \rho_2}\rangle(dcl_1) \to \tau$
Functions	$f ::= fd \mid fe$
Modules	$Mods ::= \{(id_1, f_1), ..., (id_n, f_n)\}$

Syntax of Rustlight

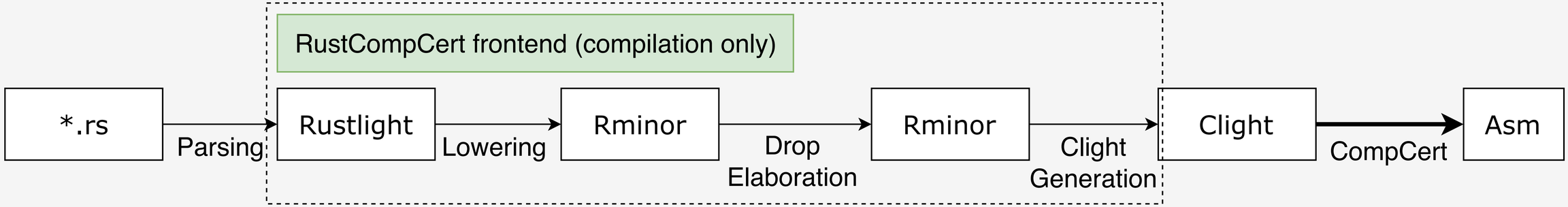
- Supported: struct and enum, heap allocation (by Box) and reference
- Unsupported: concurrency, polymorphism, Trait and higher-order functions

A verified compiler for a subset of Rust




```
// Rustlight code, after parsing
if (discriminant_eq(*l, Nil)) {
  ... // omitted
} else {
  let node : Node in {
    node = move (*l as Cons);
    if (node.val == x) {
      let _32 : List in {
        _32 = List::Nil(tt);
        *l = move _32;
      }
    } else {
      l = move node.next;
    }
  }
}
```

A verified compiler for a subset of Rust

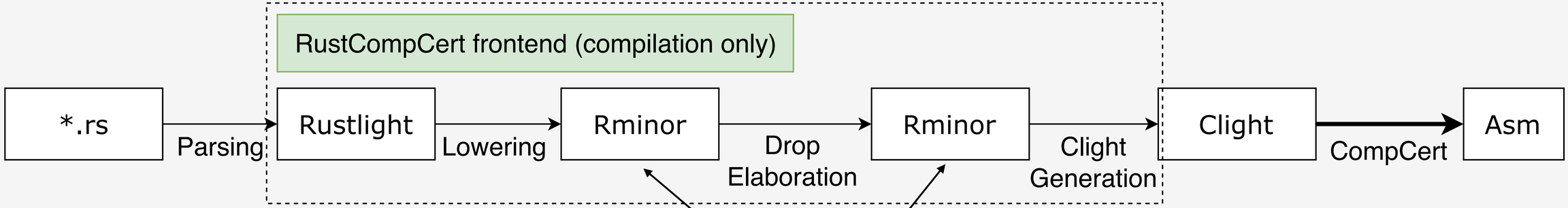



```
// Rustlight code, after parsing
if (discriminant_eq(*l, Nil)) {
  ... // omitted
} else {
  let node : Node in {
    node = move (*l as Cons);
    if (node.val == x) {
      let _32 : List in {
        _32 = List::Nil(tt);
        *l = move _32;
      }
    } else {
      l = move node.next;
    }
  }
}
```

Lowering

```
// Rminor code, after lowering
if (discriminant_eq(*l, Nil)) {
  …
} else {
  storagelive node;
  drop(node);
  node = move (*l as Cons);
  if (node.val == x) {
    storagelive _32;
    drop(_32);
    _32 = List::Nil(tt);
    drop(*l);
    *l = move _32;
    drop(_32);
    storagedead _32;
  } else {
    drop(l);
    l = move node.next;
  }
  drop(node);
  storagedead node;
}
```

A verified compiler for a subset of Rust



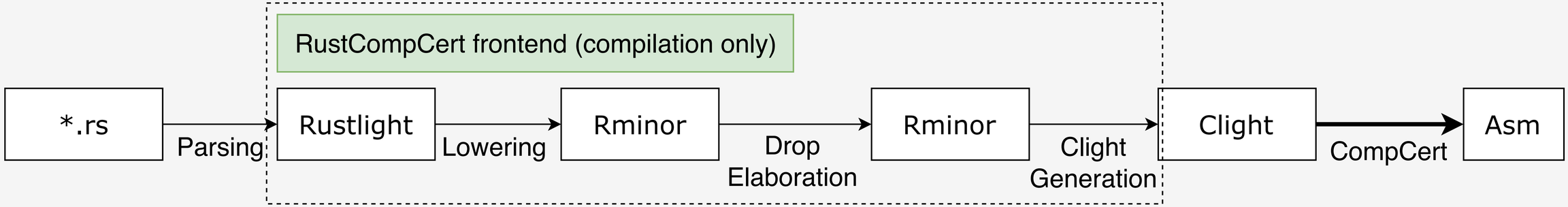

```
1.// Rustlight code, after parsing
2.if (discriminant_eq(*l, Nil)) {
3.   ... // omitted
4.} else {
5.   let node : Node in {
6.     node = move (*l as Cons);
7.     if (node.val == x) {
8.       let _32 : List in {
9.         _32 = List::Nil(tt);
10.         *l = move _32;
11.       }
12.     } else {
13.       l = move node.next;
14.     }
15.   }
16.}
```

IR of our frontend:
1. Follow the design of MIR
2. But in AST form

Lowering

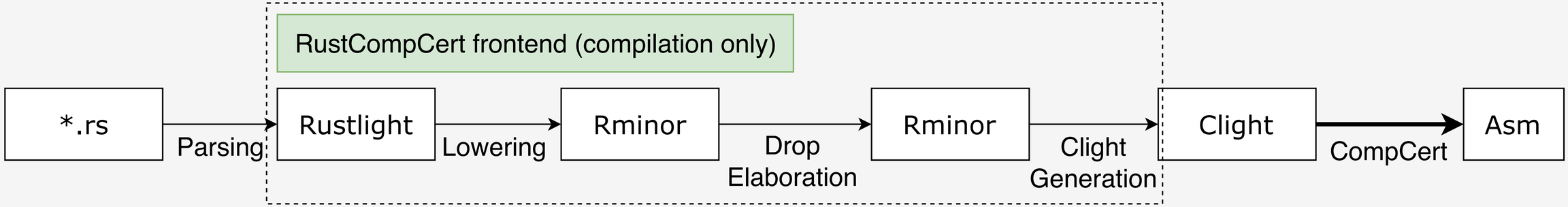
```
1.// Rminor code, after lowering
2.if (discriminant_eq(*l, Nil)) {
3.   …
4.} else {
5.   storagelive node;
6.   drop(node);
7.   node = move (*l as Cons);
8.   if (node.val == x) {
9.     storagelive _32;
10.     drop(_32);
11.     _32 = List::Nil(tt);
12.     drop(*l);
13.     *l = move _32;
14.     drop(_32);
15.     storagedead _32;
16.   } else {
17.     drop(l);
18.     l = move node.next;
19.   }
20.   drop(node);
21.   storagedead node;
22.}
```

A verified compiler for a subset of Rust



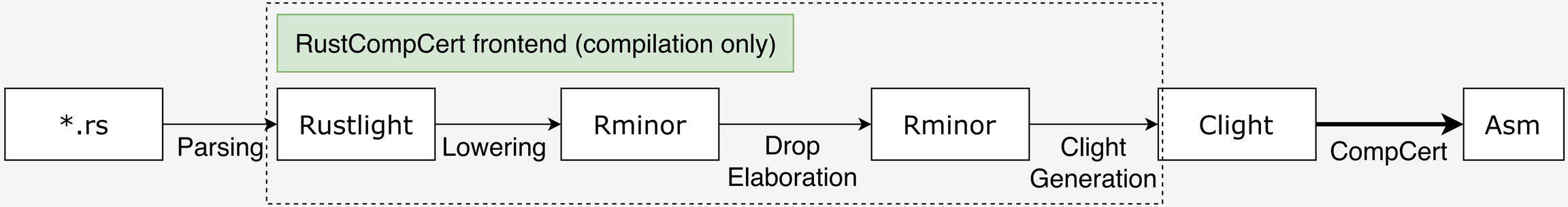

```
// Rminor code, after lowering
if (discriminant_eq(*l, Nil)) {
  …
} else {
  storagelive node;
  drop(node);
  node = move (*l as Cons);
  if (node.val == x) {
    storagelive _32;
    drop(_32);
    _32 = List::Nil(tt);
    drop(*l);
    *l = move _32;
    drop(_32);
    storagedead _32;
  } else {
    drop(l);
    l = move node.next;
  }
  drop(node);
  storagedead node;
}
```

A verified compiler for a subset of Rust




```
// Rminor code, after lowering
if (discriminant_eq(*l, Nil)) {
  …
} else {
  storagelive node;
  drop(node);
  node = move (*l as Cons);
  if (node.val == x) {
    storagelive _32;
    drop(_32);
    _32 = List::Nil(tt);
    drop(*l);
    *l = move _32;
    drop(_32);
    storagedead _32;
  } else {
    drop(l);
    l = move node.next;
  }
  drop(node);
  storagedead node;
}
```

A verified compiler for a subset of Rust

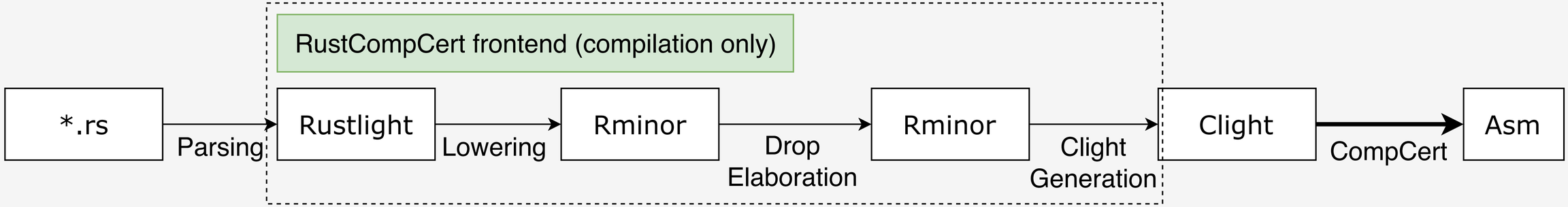



```
1.// Rminor code, after lowering
2.if (discriminant_eq(*l, Nil)) {
3.   …
4.} else {
5.   storagelive node;
6.   drop(node);
7.   node = move (*l as Cons);
8.   if (node.val == x) {
9.     storagelive _32;
10.     drop(_32);
11.     _32 = List::Nil(tt);
12.     drop(*l);
13.     *l = move _32;
14.     drop(_32);
15.     storagedead _32;
16.   } else {
17.     drop(l);
18.     l = move node.next;
19.   }
20.   drop(node);
21.   storagedead node;
22.}
```

Drop Elaboration

```
1.// Rminor code, after Drop Elaboration
2.if (node.val == x) {
3.   storagelive _32;
4.   _32 = List::Nil(tt);
5.   *l = move _32;
6.   storagedead _32;
7.} else {
8.   drop(l);
9.   __dropflag_node__field__next = 0;
10.   l = move node.next;
11.}
12.if (__dropflag_node__field__next) {
13.   __dropflag_node__field__next = 0;
14.   drop(*node.next);
15.   drop(node.next);
16.}
```

A verified compiler for a subset of Rust

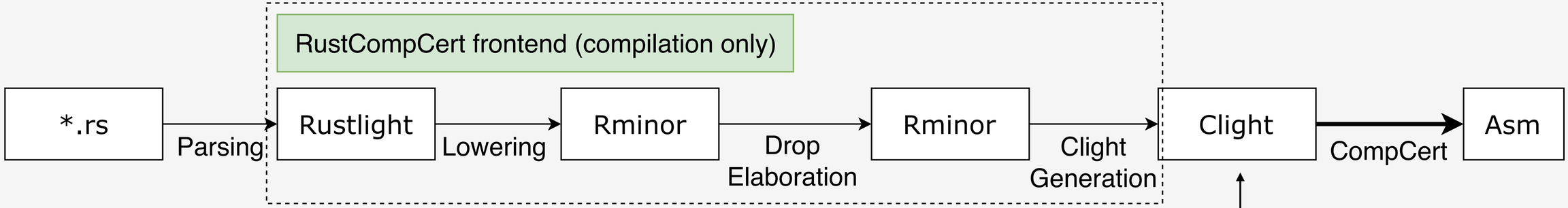


Clight Generation

```
1.Generated Clight code
2.if (node.val == x) {
3.  _34.__body_of_List.Nil = 0;
4.  _34.__tag_of_List = 0;
5.  *l = _34;
6.} else {
7.  $73 = &l;
8.  free(*$73);
9.  __dropflag_node__field__next = 0;
10.  l = node.next;
11.}
12.if (__dropflag_node__field__next) {
13.  __dropflag_node__field__next = 0;
14.  $74 = &*node.next;
15.  __drop_in_place_List($74);
16.  $75 = &node.next;
17.  free(*$75);
18.}
```

```
1.Generated drop glue code
2.void __drop_in_place_List(struct List *dropglue_param)
3.{ ... }
```

A verified compiler for a subset of Rust



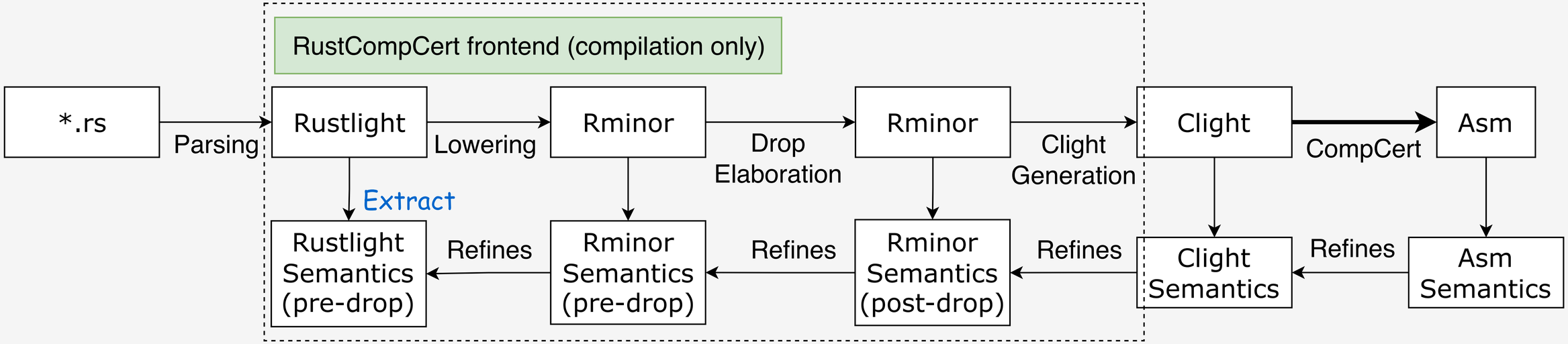
Clight Generation

```
1.Generated Clight code
2.if (node.val == x) {
3.   _34.__body_of_List.Nil = 0;
4.   _34.__tag_of_List = 0;
5.   *l = _34;
6.} else {
7.   $73 = &l;
8.   free(*$73);
9.   __dropflag_node__field__next = 0;
10.   l = node.next;
11.}
12.if (__dropflag_node__field__next) {
13.   __dropflag_node__field__next = 0;
14.   $74 = &*node.next;
15.   __drop_in_place_List($74);
16.   $75 = &node.next;
17.   free(*$75);
18.}
```

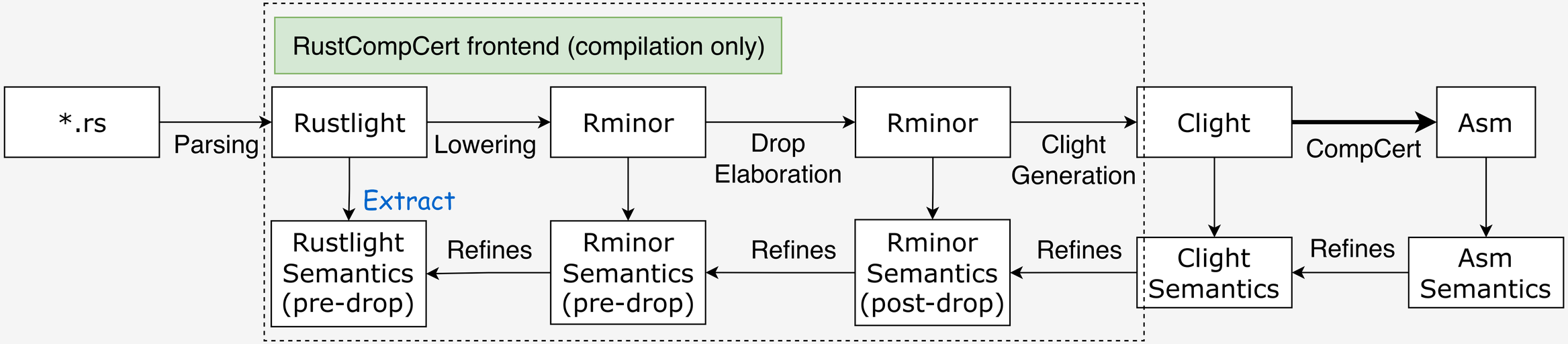
```
1.Generated drop glue code
2.void __drop_in_place_List(struct List *dropglue_param)
3.{ ... }
```

Tradeoff of choosing Clight as the target language:
- Pros: reuse most of the passes in CompCert
- Cons: reduces the extensibility of the frontend

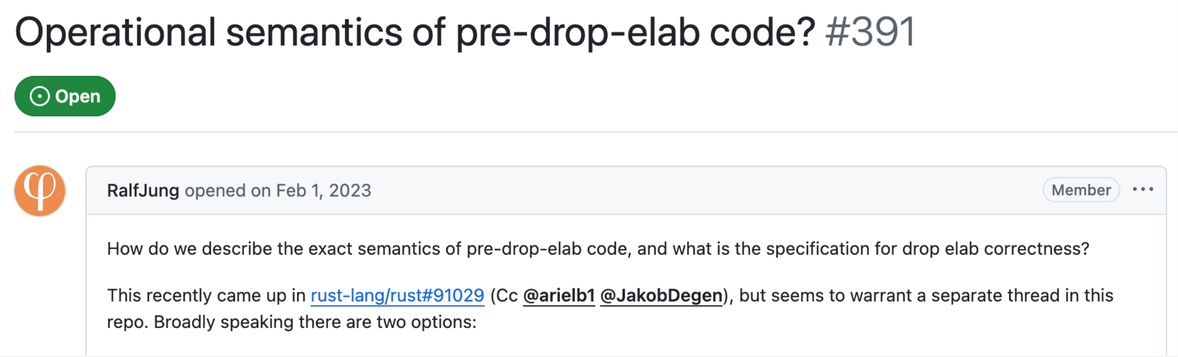
A verified compiler for a subset of Rust

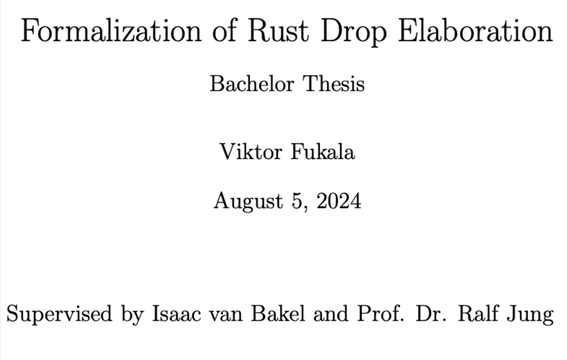


A verified compiler for a subset of Rust



- Pre-drop semantics: defines dynamic drop semantics before Drop Elaboration




Formalization of Rust Drop Elaboration

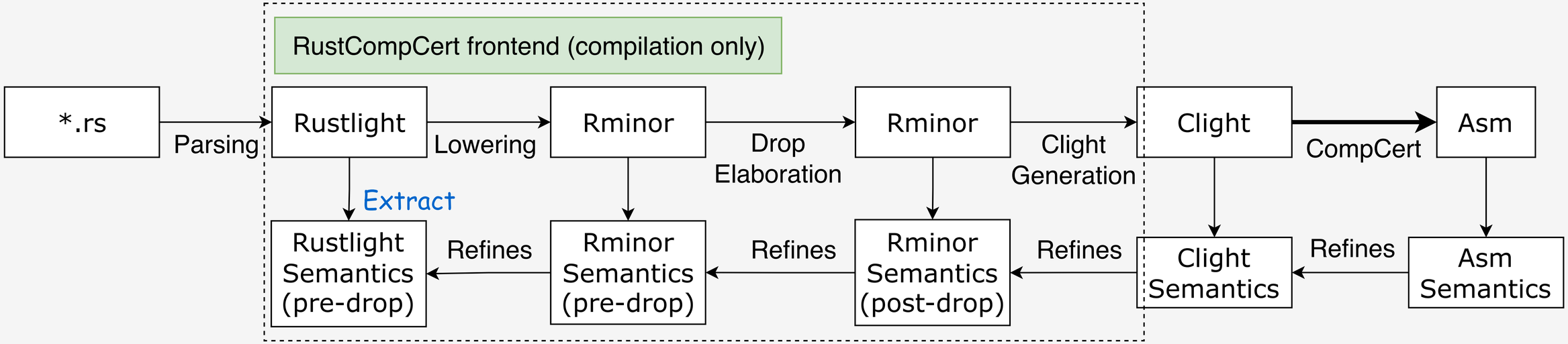
Bachelor Thesis

Viktor Fukala

August 5, 2024

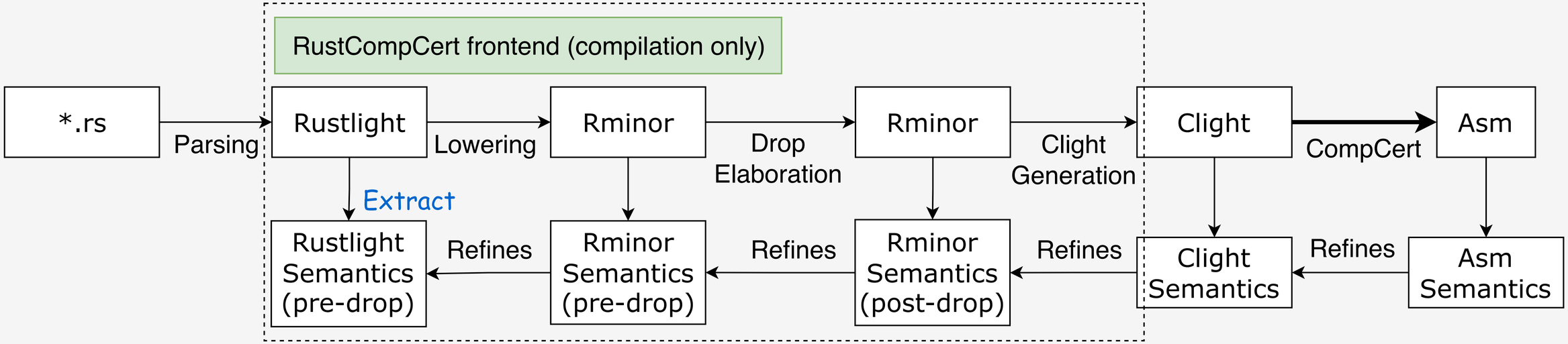
Supervised by Isaac van Bakel and Prof. Dr. Ralf Jung

A verified compiler for a subset of Rust



- Pre-drop semantics: defines dynamic drop semantics before Drop Elaboration
- CompCert memory model is enough for these compilation passes

A verified compiler for a subset of Rust



- Pre-drop semantics: defines dynamic drop semantics before Drop Elaboration
- CompCert memory model is enough for these compilation passes
- Compiler correctness: $\forall P_{\text{Rustlight}}, P_{\text{Asm}}, \text{RustCompCert}(P_{\text{Rustlight}}) = P_{\text{Asm}} \Rightarrow \text{Beh}(P_{\text{Rustlight}}) \supseteq \text{Beh}(P_{\text{Asm}})$

Formalize and verify move & borrow checking

Formalize and verify move & borrow checking

- How to formalize the program analysis in our compiler:
  - Drop elaboration and move checking: initialization analysis
  - Borrow checking: Polonius-based borrow analysis

Formalize and verify move & borrow checking

- How to formalize the program analysis in our compiler:
  - Drop elaboration and move checking: initialization analysis
  - Borrow checking: Polonius-based borrow analysis
- Reimplementing rustc’s analysis is not realistic

Formalize and verify move & borrow checking

- How to formalize the program analysis in our compiler:
  - Drop elaboration and move checking: initialization analysis
  - Borrow checking: Polonius-based borrow analysis
- Reimplementing rustc’s analysis is not realistic
  - Reason: lack of verification framework for advanced data structures used in rustc

Formalize and verify move & borrow checking

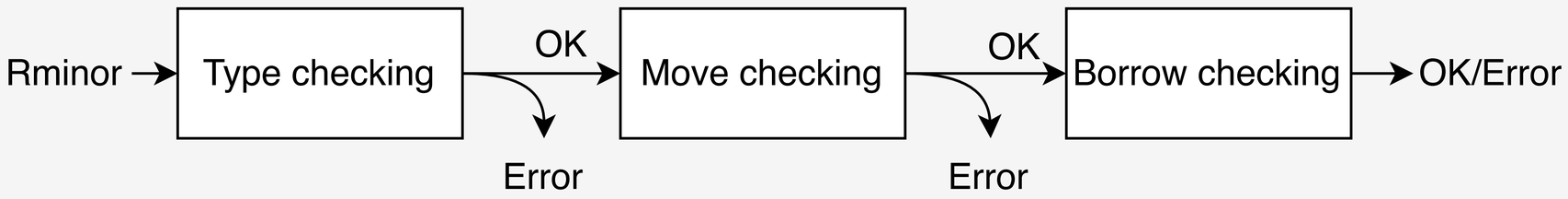
- How to formalize the program analysis in our compiler:
  - Drop elaboration and move checking: initialization analysis
  - Borrow checking: Polonius-based borrow analysis
- Reimplementing rustc's analysis is not realistic
  - Reason: lack of verification framework for advanced data structures used in rustc
- Our approach: reuse CompCert's dataflow analysis framework

Formalize and verify move & borrow checking

- How to formalize the program analysis in our compiler:
  - Drop elaboration and move checking: initialization analysis
  - Borrow checking: Polonius-based borrow analysis
- Reimplementing rustc's analysis is not realistic
  - Reason: lack of verification framework for advanced data structures used in rustc
- Our approach: reuse CompCert's dataflow analysis framework

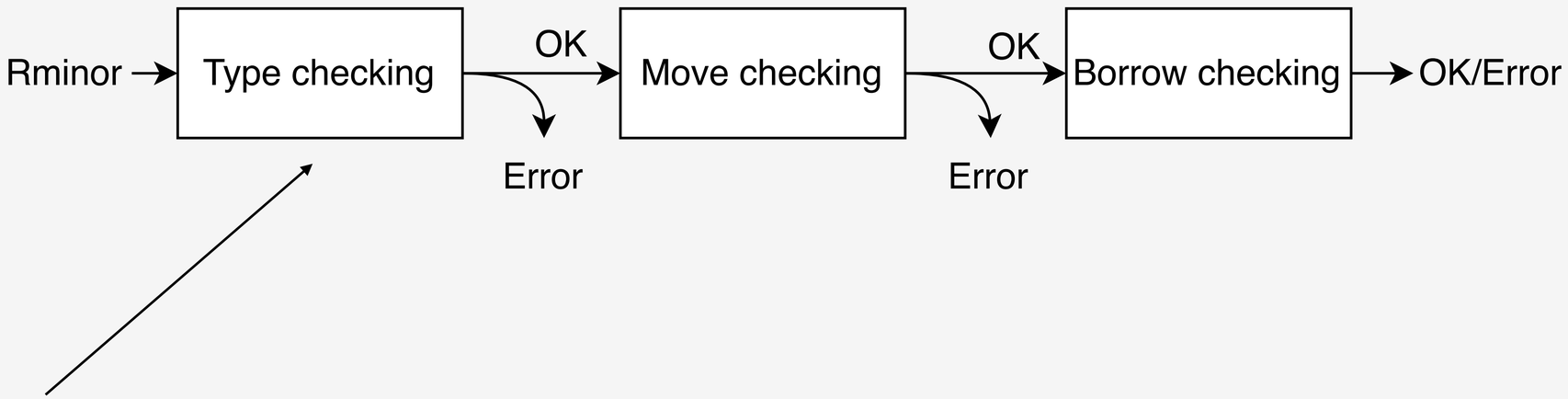
Formalize move & borrow checking

- Structure of the borrow checker in RustCompCert:



Formalize move & borrow checking

- Structure of the borrow checker in RustCompCert:

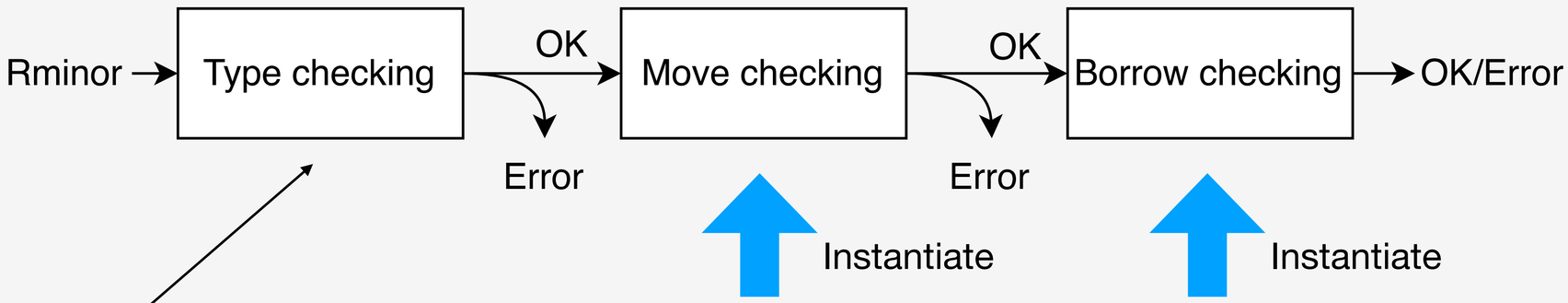


Check well typedness, e.g.,

$$\frac{\Gamma \vdash p \quad \Gamma \vdash e \quad \mathrm{typeof}(p) = \mathrm{typeof}(e)}{\Gamma \vdash p = e}$$

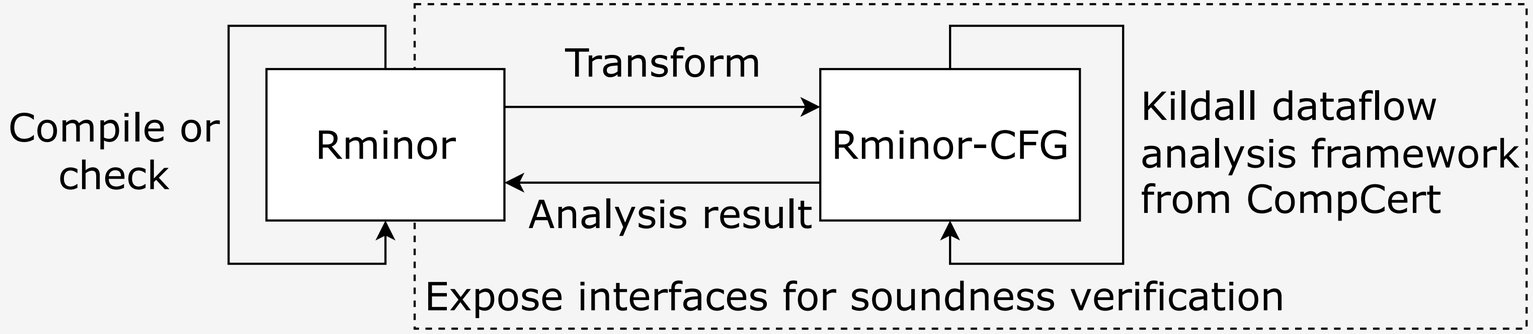
Formalize move & borrow checking

- Structure of the borrow checker in RustCompCert:



Check well typedness, e.g.,

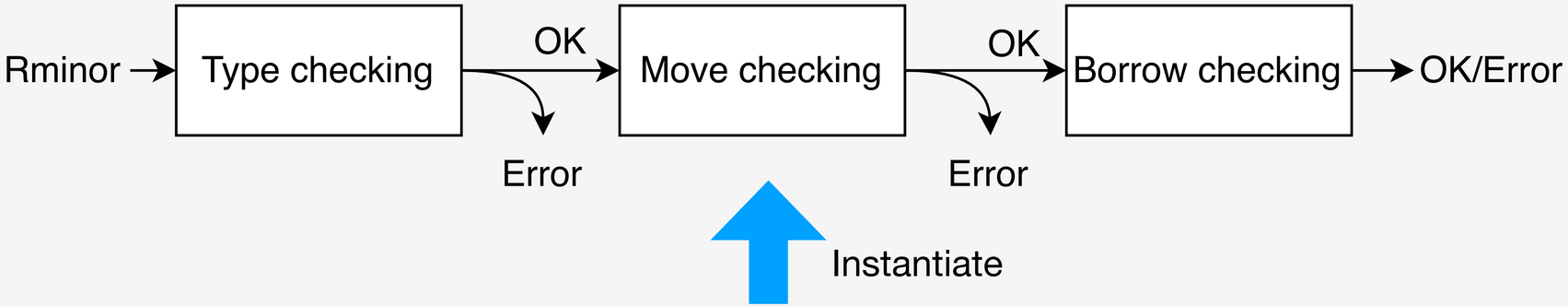
$$\frac{\Gamma \vdash p \quad \Gamma \vdash e \quad \mathrm{typeof}(p) = \mathrm{typeof}(e)}{\Gamma \vdash p = e}$$



A dataflow analysis library for Rminor

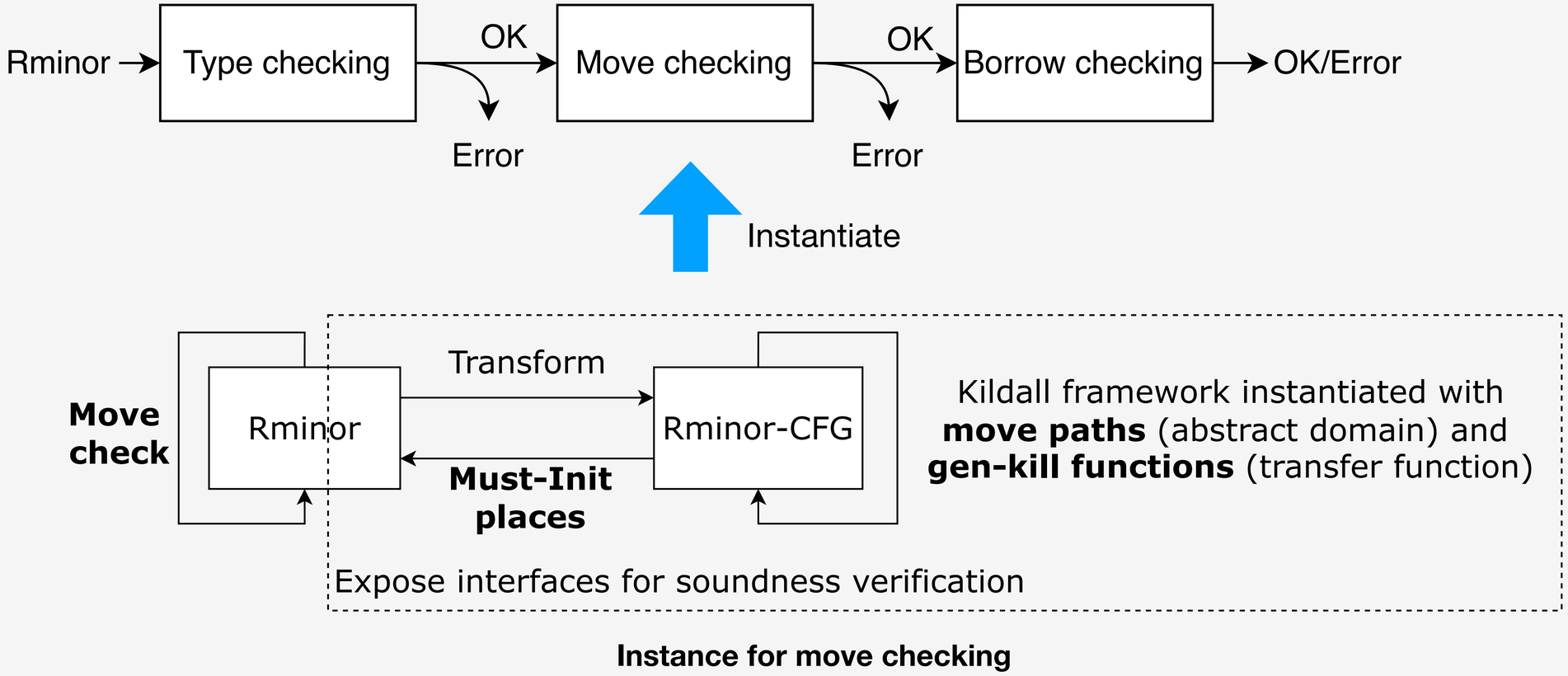
Formalize move checking

- Internal structure of the move checker in RustCompCert



Formalize move checking

- Internal structure of the move checker in RustCompCert

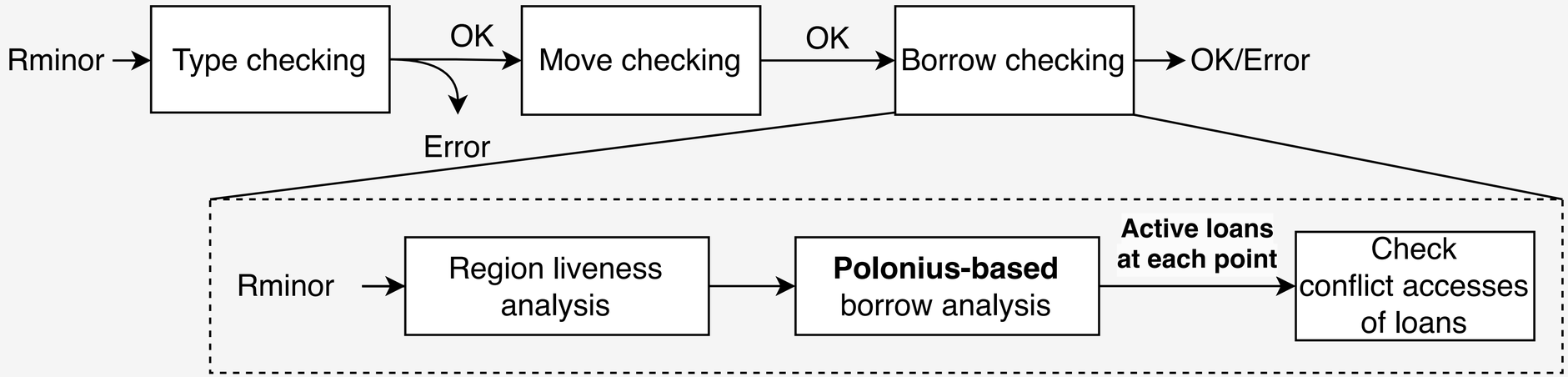


Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert

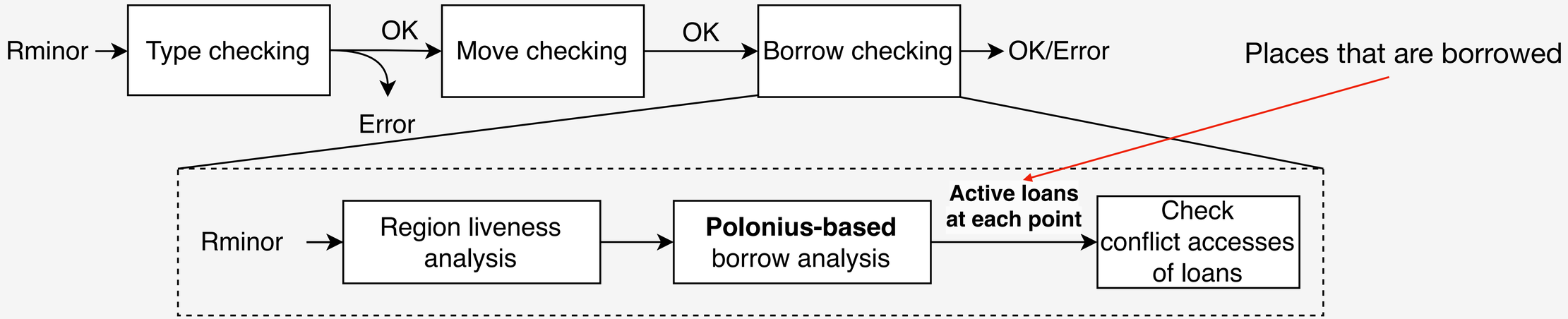
Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert



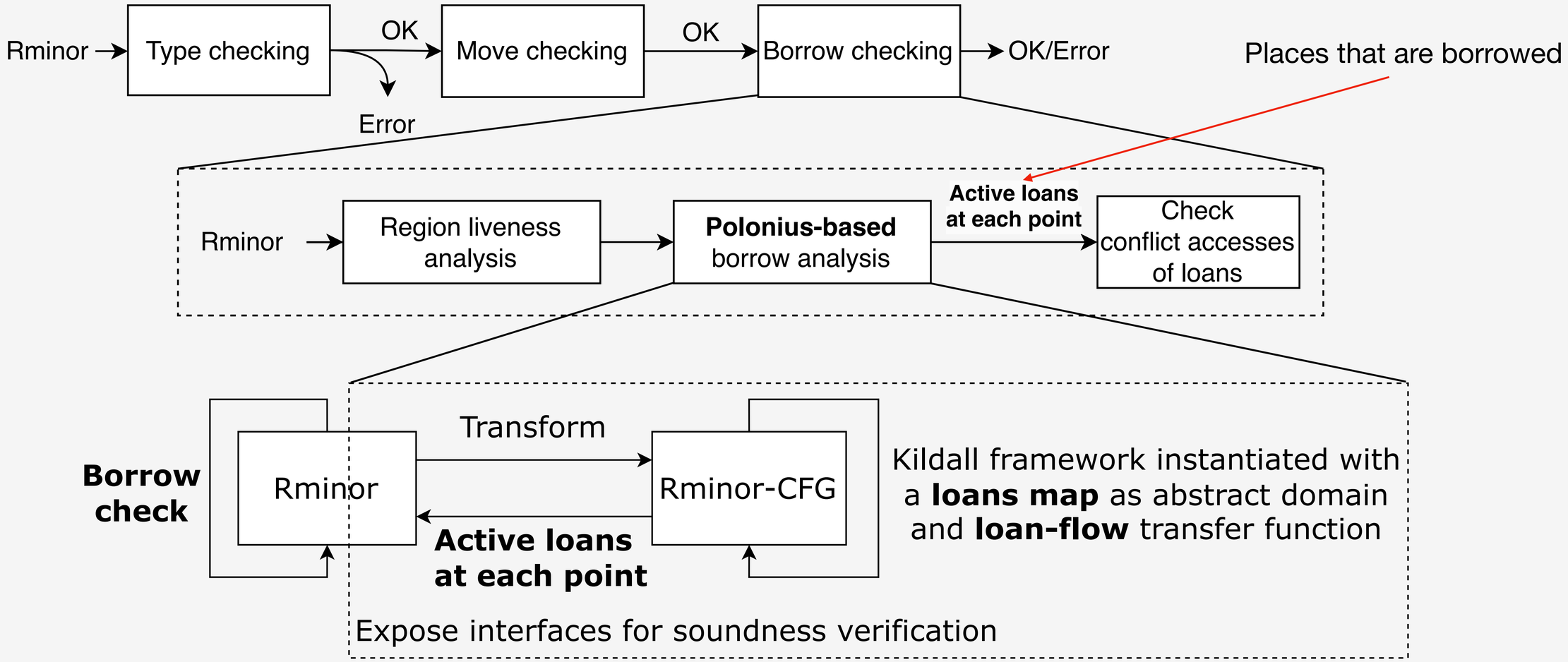
Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert



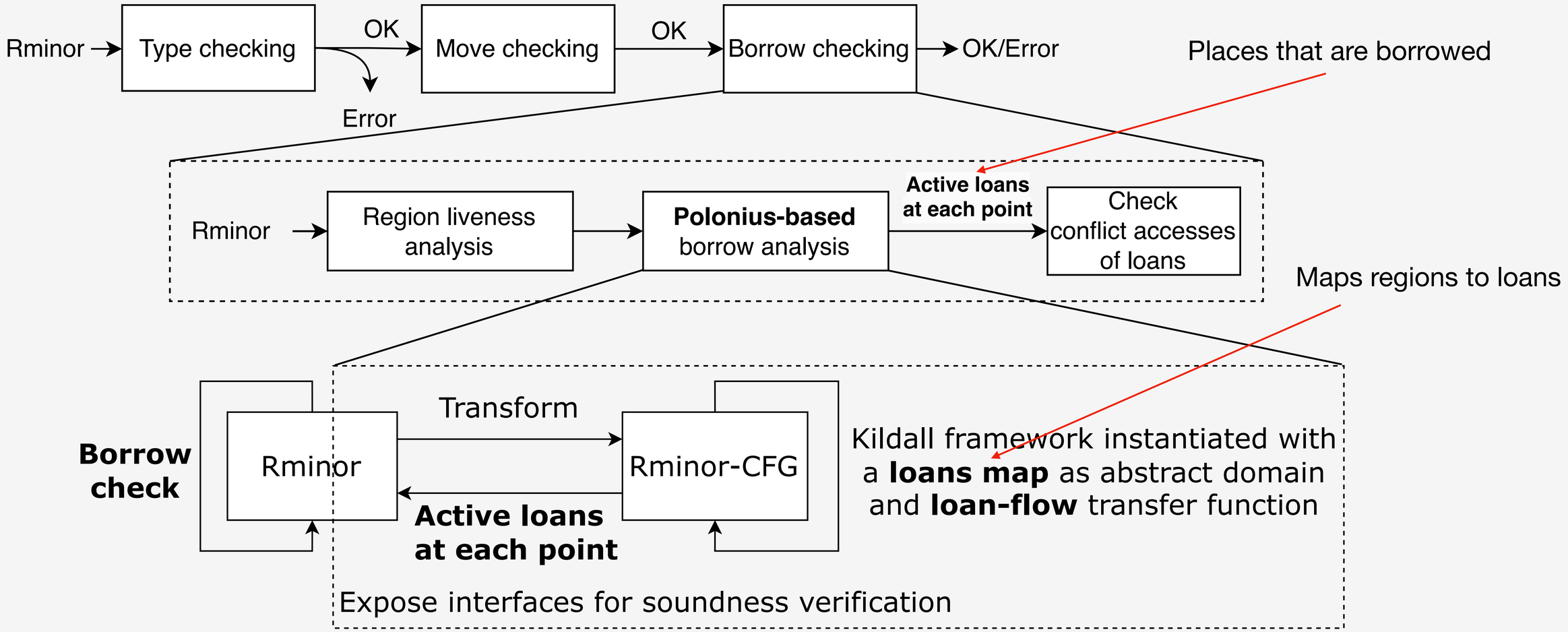
Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert



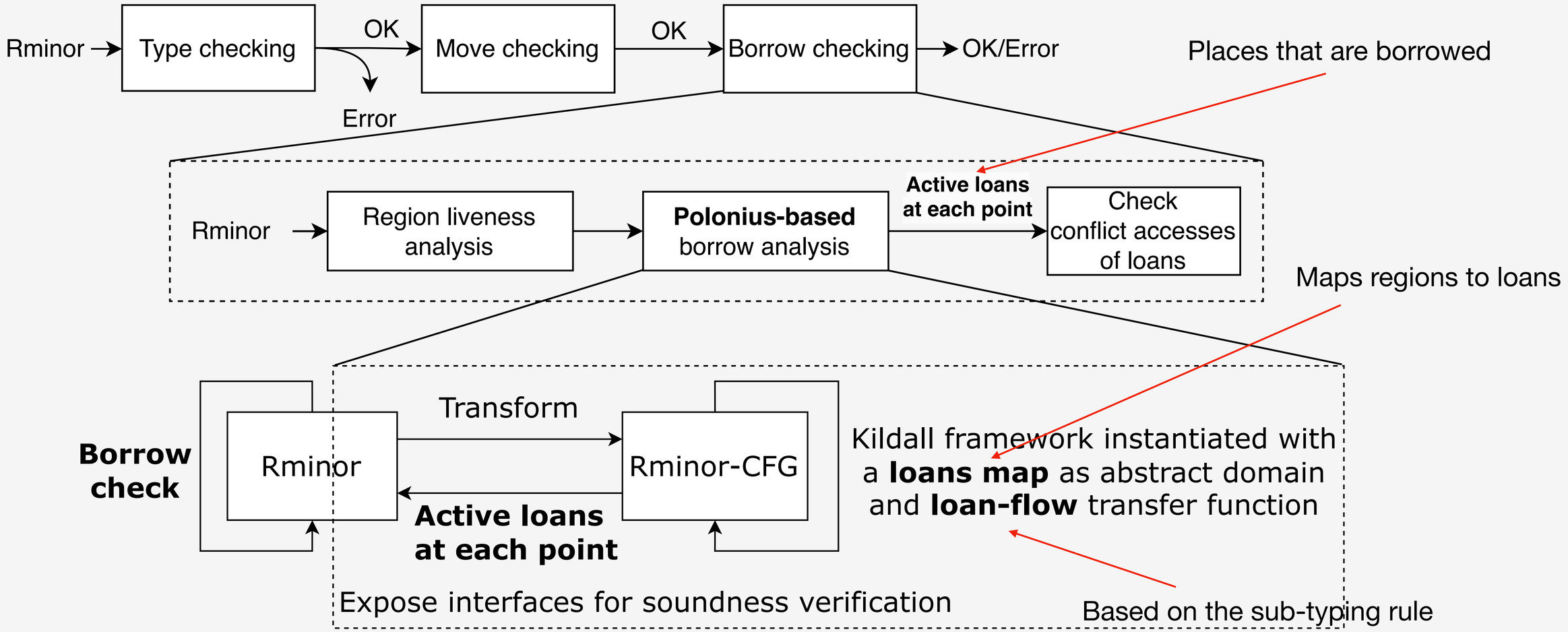
Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert



Formalize borrow checking

- Internal structure of the borrow checker in RustCompCert



Formalize and verify borrow checking

Formalize and verify borrow checking

- The verification is still in progress

Formalize and verify borrow checking

- The verification is still in progress
  - Soundness: borrow checked programs are memory safe, e.g., it refines a functional semantics

Formalize and verify borrow checking

- The verification is still in progress
  - Soundness: borrow checked programs are memory safe, e.g., it refines a functional semantics
  - Plan: based on abstract interpretation, to study what properties does the Polonius-based borrow checker approximate

Formalize and verify borrow checking

- The verification is still in progress
  - Soundness: borrow checked programs are memory safe, e.g., it refines a functional semantics
  - Plan: based on abstract interpretation, to study what properties does the Polonius-based borrow checker approximate
- Limitations of our borrow checker:

Formalize and verify borrow checking

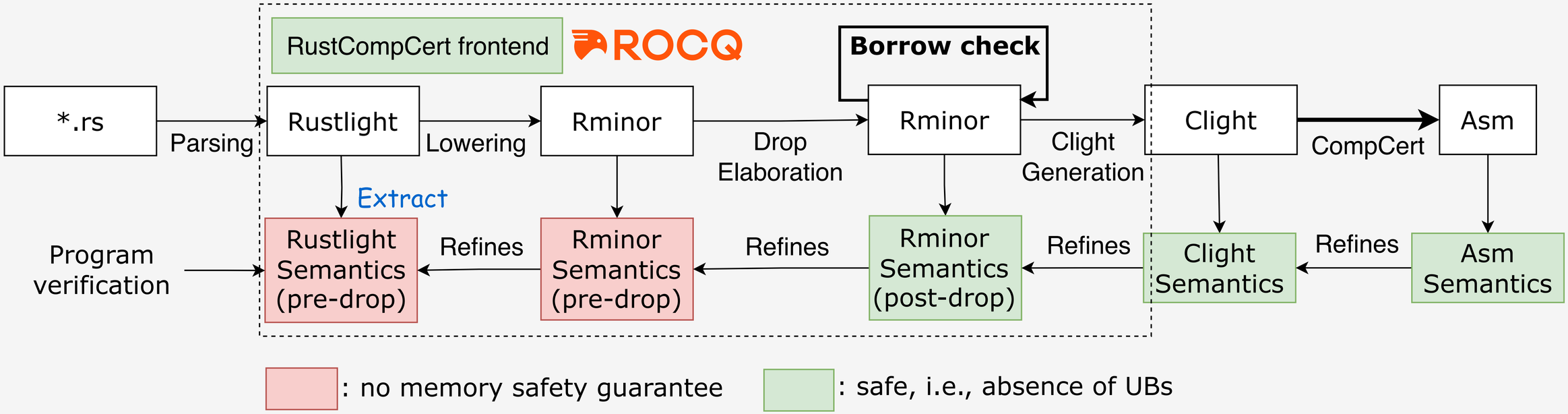
- The verification is still in progress
  - Soundness: borrow checked programs are memory safe, e.g., it refines a functional semantics
  - Plan: based on abstract interpretation, to study what properties does the Polonius-based borrow checker approximate
- Limitations of our borrow checker:
  - contravariant

Formalize and verify borrow checking

- The verification is still in progress
  - Soundness: borrow checked programs are memory safe, e.g., it refines a functional semantics
  - Plan: based on abstract interpretation, to study what properties does the Polonius-based borrow checker approximate
- Limitations of our borrow checker:
  - contravariant
  - two-phase borrow

Formalize and verify borrow checking

- Expected outcome: a **verified** and **verifying** compiler for a subset of Rust



Overview of RustCompCert

Interoperation between safe and unsafe modules

Interoperation between safe and unsafe modules

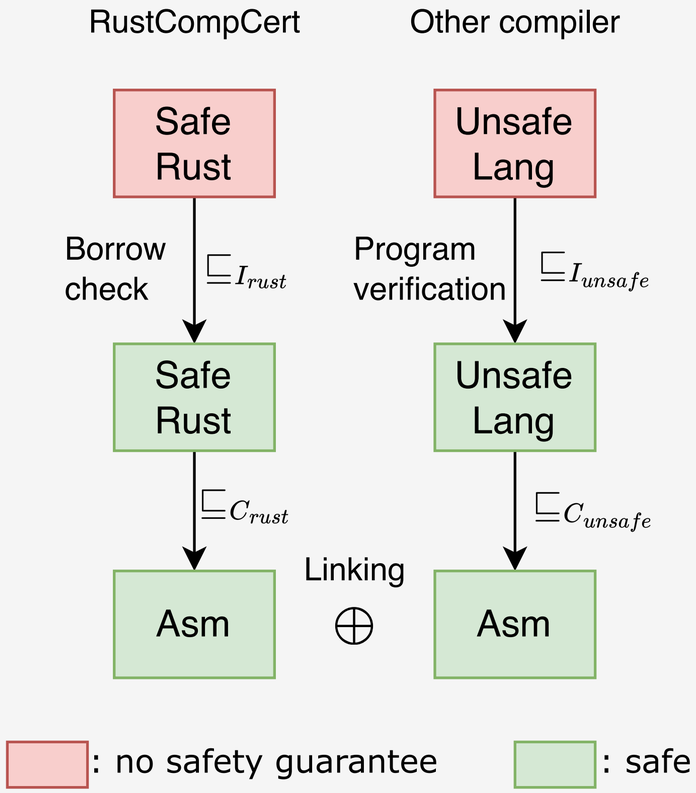
- High-level ideas: allow safe Rust to invoke unsafe code via FFI

Interoperation between safe and unsafe modules

- High-level ideas: allow safe Rust to invoke unsafe code via FFI
  - The contract between safe and unsafe is defined as **rely-guarantee conditions** — a technique used to represent contract of compositional compiler correctness

Interoperation between safe and unsafe modules

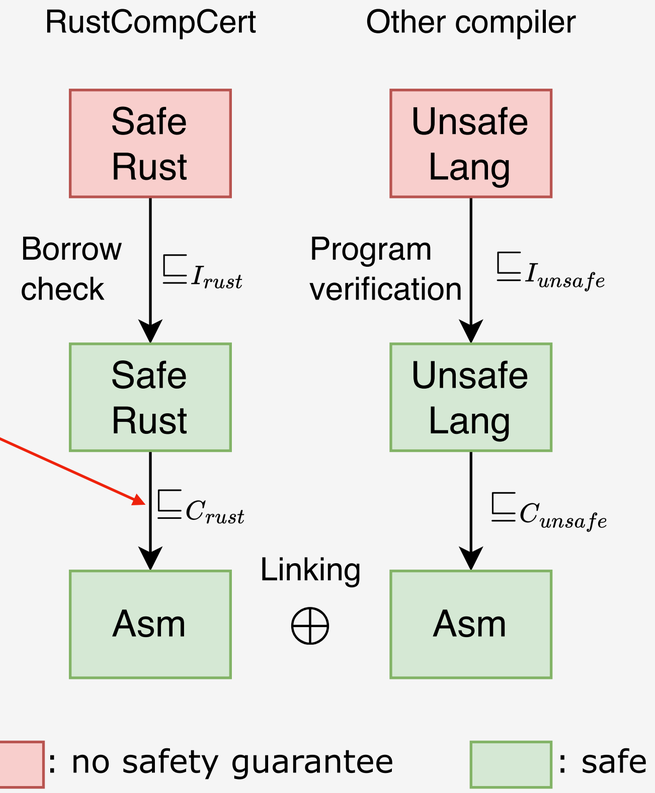
- High-level ideas: allow safe Rust to invoke unsafe code via FFI
  - The contract between safe and unsafe is defined as **rely-guarantee conditions** — a technique used to represent contract of compositional compiler correctness



Interoperation between safe and unsafe modules

- High-level ideas: allow safe Rust to invoke unsafe code via FFI
  - The contract between safe and unsafe is defined as **rely-guarantee conditions** — a technique used to represent contract of compositional compiler correctness

To compose the correctness of verified compilation , it requires $C_{rust} \equiv C_{unsafe}$

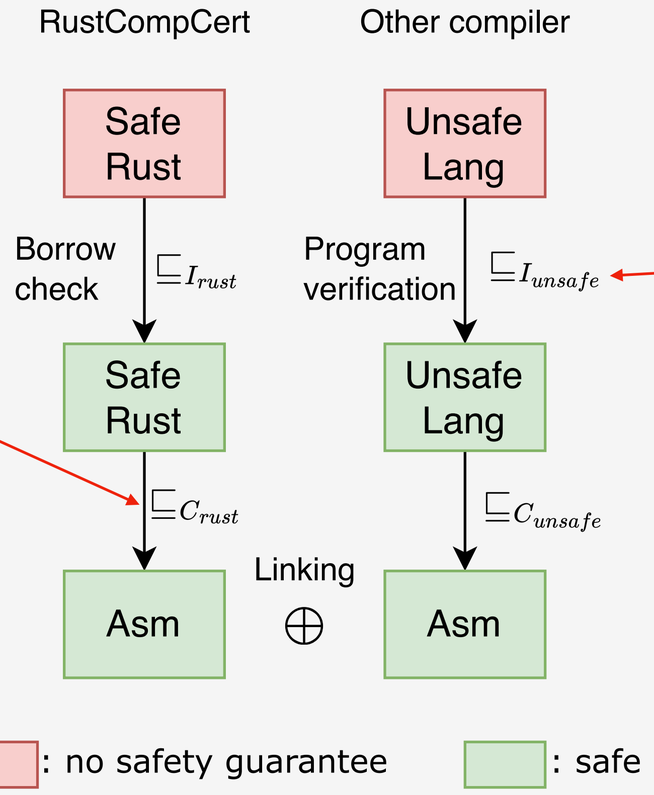


Interoperation between safe and unsafe modules

- High-level ideas: allow safe Rust to invoke unsafe code via FFI
  - The contract between safe and unsafe is defined as **rely-guarantee conditions** — a technique used to represent contract of compositional compiler correctness

To compose the correctness of verified compilation , it requires $C_{rust} \equiv C_{unsafe}$

To compose the correctness of verifying compilation, it requires $I_{rust} \equiv I_{unsafe}$



Future plan

Future plan

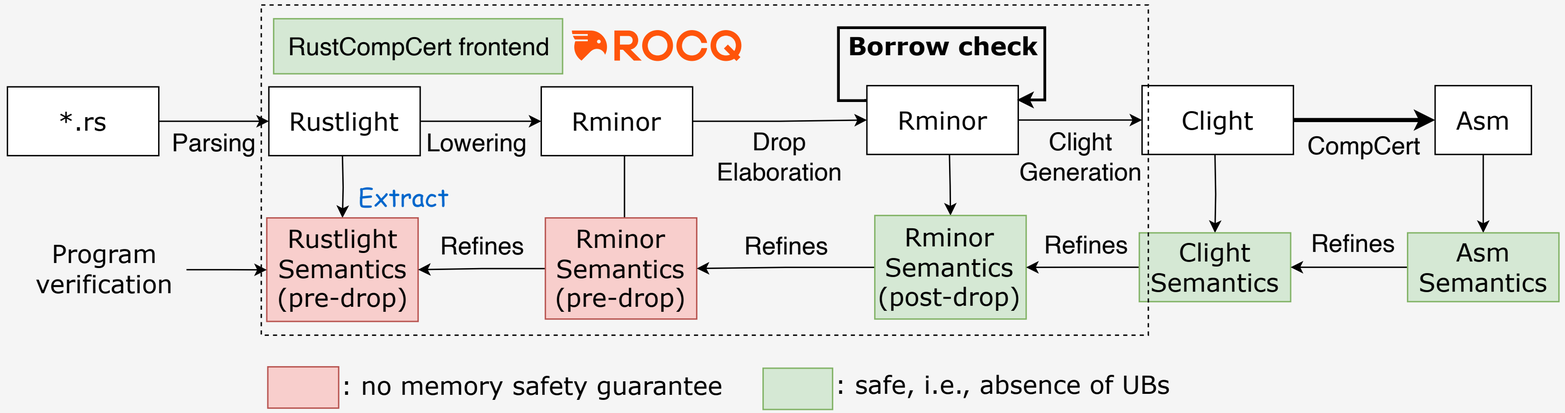
- Connect to real Rust code using tools like Charon[Ho et al. 2025]
  - Improve usability and test coverage

Future plan

- Connect to real Rust code using tools like Charon[Ho et al. 2025]
  - Improve usability and test coverage
- Connect to Rust verification tools to achieve end-to-end verification
  - E.g., establish refinement between the semantics of Rminor and LLBC[Ho et al. 2024]

Thank you for your listening!

- This talk:



Overview of RustCompCert

- Ongoing work: verify the borrow checker
- Future work:
  - Connect to real world Rust programs
  - Connect to Rust verification tools